

Database

The agents can use databases in order to read or write data. BAF allows the easy connection with external databases, thanks to the wide variety of Python libraries available, such as [SQLAlchemy](#).

BAF provides a [monitoring database](#) that can be used to monitor the agent usage.

DB Handler

Besides the [Monitoring Database](#), BAF now provides a DB handler to query relational databases from your agent logic.

The DB handler reads one or more DB credentials from the ``db.sql`` section in your agent properties file and opens connections lazily (only when a query is executed).

Initialize the DB Handler

Create the handler once when defining the agent:

```
from baf.core.agent import Agent
from baf.db import DBHandler

agent = Agent('weather_agent')
agent.load_properties('config.yaml')

# No DB connection is created yet
db_handler: DBHandler = agent.use_db_handler()
```

You can define multiple DBs in configuration:

```
db:
  sql:
    - db1:
        dialect: postgresql
        host: localhost
        port: 5432
        database: weather_db
        username: user
        password: pass
    - db2:
        dialect: sqlite
        file: path/to/local_database.db
```

Query a DB in a State Body

The handler is available in sessions as `session.db_handler`.

```
from baf.core.session import Session

def weather_body(session: Session):
    city = 'Barcelona'

    # Raw SQL query
    temperature = session.db_handler.query(
        db_name='db1',
```

```

        query=f"SELECT temperature FROM weather WHERE city='{city}'"
    )

    session.reply(f"The weather in {city} is {temperature}°C.")

```

You can also provide a natural language query and an LLM to translate it into SQL:

```

temperature = session.db_handler.query(
    db_name='db1',
    query=f"Get the weather in {city}",
    llm=gpt,
)

```

Operation-specific Queries

Besides `query()`, the DB handler provides operation-specific methods:

- `select()`: only accepts SELECT SQL
- `insert()`: only accepts INSERT SQL
- `update()`: only accepts UPDATE SQL
- `delete()`: only accepts DELETE SQL

If the SQL command does not match the called method, the query is not executed, an error is logged, and the method returns `None`. This validation also applies when SQL is generated from natural language with an LLM.

```

# SELECT
rows = session.db_handler.select('db1', 'SELECT * FROM weather')

# INSERT
session.db_handler.insert(
    'db1',
    "INSERT INTO weather(city, temperature) VALUES ('Barcelona', 21)",
)

# UPDATE
session.db_handler.update(
    'db1',
    "UPDATE weather SET temperature=22 WHERE city='Barcelona'",
)

# DELETE
session.db_handler.delete(
    'db1',
    "DELETE FROM weather WHERE city='Barcelona'",
)

```

Monitoring Database

If you would like to monitor the agent performance, to discover if it is properly recognizing the message intents or the parameters, what kind of messages are throwing the fallback state (i.e. when no intent is recognized), the confidence on the intent predictions,... or gather some information

about the users, like what are the most frequent questions or how many interactions they need in order to achieve their goal when using the agent...

You can rely on a database to store usage information for later monitoring and analysis!

BAF agents have a [MonitoringDB](#) attribute (optionally used) in charge of managing the DB connection and the data insertion. When running the agent, right after its training, it connects to the DB ([MonitoringDB.connect_to_db\(\)](#)) and initializes it, creating the tables if they don't exist ([MonitoringDB.initialize_db\(\)](#)). This process is hidden from the user. To activate it, you simply need to define the [configuration properties](#) to properly connect to the database, BAF is in charge of the rest.

Database Schema

This section describes the followed database schema, with a description of each table in the database.

Note that the creation of the tables is automatic. This section is purely informative. You may need this information in order to retrieve data for the agent monitoring analysis.

Table session

This table stores a record for each new session.

Table schema (PostgreSQL):

```
CREATE TABLE IF NOT EXISTS public.session
(
    id INTEGER NOT NULL DEFAULT nextval('session_id_seq1'::regclass),
    agent_name CHARACTER VARYING NOT NULL,
    session_id CHARACTER VARYING NOT NULL,
    session_name CHARACTER VARYING,
    username CHARACTER VARYING,
    platform_name CHARACTER VARYING NOT NULL,
    "timestamp" TIMESTAMP without time zone NOT NULL,
    variables CHARACTER VARYING,
    CONSTRAINT session_pkey PRIMARY KEY (id),
    CONSTRAINT session_agent_name_session_id_key UNIQUE (agent_name, session_id)
)
```

Example table entries:

id	agent_name	session_id	session_name	username	platform_name	timestamp	variables
1	greetings-agent	aaddaab5-o065-40f4-a996-b584d63b0k0d	Saying hello to my agent	john	WebSocketPlatform	2024-05-02 14:52:47	{“name”: “John”, “age”: “30”}
2	greetings-agent	6642498531	Saying hello to my agent	john	TelegramPlatform	2024-05-02 14:53:46	{}

id	agent_name	session_id	session_name	username	platform_name	timestamp	variables
3	weather_agent	6ff6cf75-d6ea-495b-a465-fe4856e1b5f9	Ask weather	john	WebSocketPlatform	2024-05-02 14:53:50	

id	agent_name	session_id	session_name	username	platform_name	timestamp	variables
1	greetings_agent	aaddaab5-o065-40f4-a996-b584d63b0k0d	Saying hello to my agent	john	WebSocketPlatform	2024-05-02 14:52:47	{“name”: “John”, “age”: “30”}
2	greetings_agent	6642498531	Saying hello to my agent	john	TelegramPlatform	2024-05-02 14:53:46	{}
3	weather_agent	6ff6cf75-d6ea-495b-a465-fe4856e1b5f9	Ask weather	john	WebSocketPlatform	2024-05-02 14:53:50	{}

In this example, there are 3 different sessions recorded into the database, where the 2 first are from an agent called *greetings_agent* and the 3rd from a *weather_agent*. The platform of the session and the creation time (*timestamp* column) are also stored in the database.

Table chat

This table stores all conversations, where each row belongs to a message.

Some components may need this table in order to retrieve the chat history of a session (see [LLM](#) or [RAG](#))

Table schema (PostgreSQL):

```
CREATE TABLE IF NOT EXISTS public.chat
(
    id INTEGER NOT NULL DEFAULT nextval('session_id_seq1'::regclass),
    session_id INTEGER NOT NULL,
    type CHARACTER VARYING NOT NULL,
    content CHARACTER VARYING NOT NULL,
    is_user BOOLEAN NOT NULL,
    "timestamp" TIMESTAMP without time zone NOT NULL,
    CONSTRAINT chat_pkey PRIMARY KEY (id),
    CONSTRAINT chat_session_id_fkey FOREIGN KEY (session_id)
        REFERENCES public.session (id) MATCH SIMPLE
)
```

Example table entries:

	id	session_id	type	content	is_user	timestamp
1	1		str	Hello	True	2024-05-02 14:52:47
2	1		str	Hi! How can I assist you today?	False	2024-05-02 14:52:50
3	1		str	I want to book a flight	True	2024-05-02 14:52:59
4	2		str	Welcome to the shop! How can I assist you?	False	2024-05-02 16:22:20

Table event

This table stores all received events on agents, where each row belongs to an event.

Table schema (PostgreSQL):

```
CREATE TABLE IF NOT EXISTS public.event
(
    id INTEGER NOT NULL DEFAULT nextval('session_id_seq1'::regclass),
    session_id INTEGER,
    type CHARACTER VARYING NOT NULL,
    content CHARACTER VARYING NOT NULL,
    is_user BOOLEAN NOT NULL,
    "timestamp" TIMESTAMP without time zone NOT NULL,
    CONSTRAINT chat_pkey PRIMARY KEY (id),
    CONSTRAINT chat_session_id_fkey FOREIGN KEY (session_id)
        REFERENCES public.session (id) MATCH SIMPLE
)
```

Example table entries:

	id	session_id	event	info	timestamp
1	1		receive_message_text	Hello	2024-05-02 14:52:47
2	1		receive_message_text	Good	2024-05-02 14:52:50
3	1		receive_file	report.pdf	2024-05-02 14:52:59

Each event (row) references to its user session (the corresponding entry in the session table).

If an event was broadcasted to all agent sessions, the session_id is left empty.

Extra information of an event is stored in the info column (e.g., the received message in receive_message_text events or the file name in receive_file events)

Table transition

Every time a user [transitions](#) from one agent state to another, a new record is inserted into this table, keeping track of the followed paths within the agent's state machine.

Each transition contains the source and destination state names and the name of the event and/or condition that triggered it.

Table schema (PostgreSQL):

```
CREATE TABLE IF NOT EXISTS public.transition
(
    id INTEGER NOT NULL DEFAULT nextval('transition_id_seq'::regclass),
    session_id INTEGER NOT NULL,
    source_state CHARACTER VARYING NOT NULL,
    dest_state CHARACTER VARYING NOT NULL,
    event CHARACTER VARYING,
    condition CHARACTER VARYING,
    "timestamp" TIMESTAMP without time zone NOT NULL,
    CONSTRAINT transition_pkey PRIMARY KEY (id),
    CONSTRAINT transition_session_id_fkey FOREIGN KEY (session_id)
        REFERENCES public.session (id) MATCH SIMPLE
)
```

Example table entries:

id	session_id	source_state	dest_state	event	condition	timestamp
1	1	init_state	hello_state	receive_message_text	Intent Matching - hello_intent	2024-05-02 14:53:57
2	1	hello_state	good_state	receive_message_text	Intent Matching - good_intent	2024-05-02 14:54:25

Each transition (row) references to its user session (the corresponding entry in the *session* table). The timestamp column indicates the exact moment when the transition happened.

Table intent_prediction

Every user message goes through the [intent_classification](#) process. This table stores all user messages together with the intent predictions. This information can be then used to analyse the agent performance.

Table schema (PostgreSQL):

```
CREATE TABLE IF NOT EXISTS public.intent_prediction
(
    id INTEGER NOT NULL DEFAULT nextval('intent_prediction_id_seq1'::regclass),
    session_id INTEGER NOT NULL,
    message CHARACTER VARYING NOT NULL,
    "timestamp" TIMESTAMP without time zone NOT NULL,
    intent_classifier CHARACTER VARYING NOT NULL,
    intent CHARACTER VARYING NOT NULL,
    score DOUBLE PRECISION NOT NULL,
    CONSTRAINT intent_prediction_pkey PRIMARY KEY (id),
    CONSTRAINT intent_prediction_session_id_fkey FOREIGN KEY (session_id)
        REFERENCES public.session (id) MATCH SIMPLE
)
```

Example table entries:

id	session_id	message	timestamp	intent_classifier	intent	score
1	1	hi	2024-05-02 14:53:57	SimpleIntentClassifierTorch	hello_intent	0.9
2	1	good	2024-05-02 14:54:25	SimpleIntentClassifierTorch	good_intent	1.0
3	2	Welcome!	2024-05-02 15:57:01	SimpleIntentClassifierTorch	fallback_intent	0.7
4	3	What is the weather in Lux and Bcn?	2024-05-02 19:23:06	SimpleIntentClassifierTorch	weather_intent	0.9

Each intent prediction (row) references to its user session (the corresponding entry in the *session* table). The timestamp of the prediction and the confidence score are also stored.

Table parameter

This table stores the recognized parameters from every intent prediction (process done by the [NER](#) component of the agent). Each recognized parameter references to its intent prediction (the corresponding entry in the *intent_prediction* table). Note that there can be several parameters referencing to the same intent prediction.

Table schema (PostgreSQL):

```
CREATE TABLE IF NOT EXISTS public.parameter
(
    id INTEGER NOT NULL DEFAULT nextval('parameter_id_seq1'::regclass),
    intent_prediction_id INTEGER NOT NULL,
    name CHARACTER VARYING NOT NULL,
    value CHARACTER VARYING,
    info CHARACTER VARYING,
    CONSTRAINT parameter_pkey PRIMARY KEY (id),
    CONSTRAINT parameter_intent_prediction_id_fkey FOREIGN KEY
(intent_prediction_id)
REFERENCES public.intent_prediction (id) MATCH SIMPLE
)
```

Example table entries:

id	intent_prediction_id	name	value	info
1	4	city1	Luxembourg	
2	4	city2	Barcelona	